

Exam 2, Convex Optimization

Departamento de Matemáticas, Universidad de los Andes
Colombia

Student's name:	Luis Alejandro Rubiano
Student's code:	202013482
Email address:	la.rubiano@uniandes.edu.co
Class:	Convex Optimization
Instructor:	Mauricio Junca

Part I

- 1 **Ex. 11.13. Foundations of Optimization, Güler. (von Neumann)**
In this problem, the famous *von Neumann minimax theorem* for bimatrix games will be proved using linear programming duality. This result, which von Neumann proved in 1928 by advanced methods (Brouwer's fixed point theorem), was the starting point of game theory.

Let A be an $m \times n$ matrix and denote by Δ_{m-1} and Δ_{n-1} the unit simplices, where

$$\Delta_{n-1} = \{x \in \mathbb{R}^n : x \geq 0, \sum_{i=1}^n x_i = 1\},$$

and where Δ_{m-1} is defined similarly. The von Neumann minimax theorem states that

$$\min_{x \in \Delta_{n-1}} \max_{y \in \Delta_{m-1}} \langle Ax, y \rangle = \max_{y \in \Delta_{m-1}} \min_{x \in \Delta_{n-1}} \langle Ax, y \rangle.$$

Denote by a^i and a_j the i -th row and j -th column of A , respectively.

- (a) Show that

$$\max_{y \in \Delta_{m-1}} \langle Ax, y \rangle = \max_{1 \leq i \leq m} \langle a^i, x \rangle,$$

so that

$$\min_{x \in \Delta_{n-1}} \max_{y \in \Delta_{m-1}} \langle Ax, y \rangle = \min_{x \in \Delta_{n-1}} \max_{1 \leq i \leq m} \langle a^i, x \rangle.$$

Proof: Fix $x \in \Delta_{n-1}$, i.e., $x \geq 0$ and $\sum_{j=1}^n x_j = 1$. For any $y \in \Delta_{m-1}$, we have:

$$\langle Ax, y \rangle = \sum_{i=1}^m y_i \langle a^i, x \rangle.$$

Since $y \in \Delta_{m-1}$, it is a convex combination of the vectors a^i , so

$$\langle Ax, y \rangle \text{ is a convex combination of } \{\langle a^1, x \rangle, \dots, \langle a^m, x \rangle\}.$$

Maximizing over all such y places all the weight on the largest $\langle a^i, x \rangle$, thus

$$\max_{y \in \Delta_{m-1}} \langle Ax, y \rangle = \max_{1 \leq i \leq m} \langle a^i, x \rangle.$$

Therefore,

$$\min_{x \in \Delta_{n-1}} \max_{y \in \Delta_{m-1}} \langle Ax, y \rangle = \min_{x \in \Delta_{n-1}} \max_{1 \leq i \leq m} \langle a^i, x \rangle. \quad \square$$

(b) Show that the right-hand side of the equation above can be written as a linear program

$$\min \{z : \langle a^i, x \rangle \leq z, i = 1, \dots, m, \sum_{j=1}^n x_j = 1, x \geq 0\},$$

in the variables (x, z) .

Proof: We want to solve

$$\min_{x \in \Delta_{n-1}} \max_{1 \leq i \leq m} \langle a^i, x \rangle.$$

Introduce a variable z to represent the maximum over i , yielding the equivalent problem:

$$\min_{x, z} \{z : \langle a^i, x \rangle \leq z \text{ for all } i, \sum_{j=1}^n x_j = 1, x \geq 0\}.$$

This is clearly a linear program in (x, z) . $\square$

(c) Show that the dual of the linear program above is

$$\max \{ \delta : \sum_{i=1}^m y_i a^i - \delta e \geq 0, \sum_{i=1}^m y_i = 1, y \geq 0 \},$$

in the variables (y, δ) . Here $e \in \mathbb{R}^n$ is the vector with all components 1. *Hint:* One way to proceed is to write the Lagrangian function

$$L(x, z; y, \delta) = z + \sum_{i=1}^m y_i (\langle a^i, x \rangle - z) + \delta (1 - \langle e, x \rangle)$$

on the set $X \times \Lambda$, where $X = \{x : x \geq 0\}$ and $\Lambda = \{(y, \delta) : y \geq 0\}$, and to formulate the dual using L .

Proof: Consider the primal LP:

$$\begin{aligned} \min \quad & z \\ \text{subject to} \quad & \langle a^i, x \rangle \leq z, \quad i = 1, \dots, m, \\ & \sum_{j=1}^n x_j = 1, \\ & x \geq 0. \end{aligned}$$

The Lagrangian for this problem with multipliers $y_i \geq 0$ for $\langle a^i, x \rangle - z \leq 0$ and a multiplier δ for the equality $\sum_j x_j = 1$ is:

$$\begin{aligned} L(x, z; y, \delta) &= z + \sum_{i=1}^m y_i (\langle a^i, x \rangle - z) + \delta (1 - \langle e, x \rangle) \\ &= z(1 - \sum_{i=1}^m y_i) + \sum_{i=1}^m y_i \langle a^i, x \rangle + \delta (1 - \sum_{j=1}^n x_j). \end{aligned}$$

For x -directions, to ensure boundedness, we must have

$$\sum_{i=1}^m y_i a^i - \delta e \geq 0.$$

For z , we need $1 - \sum_{i=1}^m y_i = 0$, i.e., $\sum_{i=1}^m y_i = 1$.

Thus, the dual problem is:

$$\max_{\delta, y} \{ \delta : \sum_{i=1}^m y_i a^i - \delta e \geq 0, \sum_{i=1}^m y_i = 1, y \geq 0 \}. \quad \square$$

(d) Using the already encountered techniques in parts (a) and (b), show that the dual linear program is equivalent to the right-hand side of the von Neumann minimax theorem.

Proof: From the dual form:

$$\max_{\delta, y} \{ \delta : \sum_{i=1}^m y_i a^i - \delta e \geq 0, \sum_{i=1}^m y_i = 1, y \geq 0 \},$$

the condition $\sum_{i=1}^m y_i a^i - \delta e \geq 0$ implies that for every $x \in \Delta_{n-1}$,

$$\langle \sum_{i=1}^m y_i a^i, x \rangle \geq \delta.$$

Since $\sum_{i=1}^m y_i = 1$, we have $y \in \Delta_{m-1}$. Thus,

$$\min_{x \in \Delta_{n-1}} \langle Ax, y \rangle \geq \delta.$$

Maximizing δ is then equivalent to maximizing

$$\min_{x \in \Delta_{n-1}} \langle Ax, y \rangle$$

over $y \in \Delta_{m-1}$.

Therefore, the dual problem is

$$\max_{y \in \Delta_{m-1}} \min_{x \in \Delta_{n-1}} \langle Ax, y \rangle. \quad \square$$

2 Ex. 11.15. Foundations of Optimization, Güler. (Teboulle) Consider the minimization problem

$$\begin{aligned} \min & - \sum_{i=1}^n \ln x_i \\ \text{s.t.} & \frac{1}{2} \langle Qx, x \rangle + \langle b, x \rangle + c \leq 0, \end{aligned}$$

where Q is an $n \times n$ symmetric positive definite matrix. Assume that the constraint set contains a vector $\bar{x} > 0$ such that $\frac{1}{2} \langle Q\bar{x}, \bar{x} \rangle + \langle b, \bar{x} \rangle + c < 0$.

(a) Show that (P) has a minimizer.

Proof:

$$(P) \quad \min_{x > 0} - \sum_{i=1}^n \ln(x_i) \quad \text{s.t.} \quad \frac{1}{2} \langle Qx, x \rangle + \langle b, x \rangle + c \leq 0.$$

Since $-\sum_{i=1}^n \ln(x_i)$ is strictly convex and tends to $+\infty$ as any $x_i \rightarrow 0^+$, it enforces that any minimizing sequence cannot approach the boundary $x_i = 0$. Thus, the minimizer cannot lie on the boundary of the positive orthant. The level sets of the objective function are compact in the interior of the positive orthant. Since the feasible set

$$F = \{x > 0 \mid \frac{1}{2} \langle Qx, x \rangle + \langle b, x \rangle + c \leq 0\}$$

contains a strictly feasible point $\bar{x} > 0$ and Q is positive definite, it follows that F is a closed and convex set with a nonempty interior. Since the objective function is continuous and strictly convex, and the feasible set is nonempty, convex, and closed, the problem (P) has a minimizer. $\square$

(b) The direct approach to formulating the dual will lead to a problem that is hard to write down explicitly. The following trick will lead to a manageable dual problem. Since Q is positive semidefinite, it has a square root, that is, an $n \times n$ positive semidefinite matrix A such that $A^2 = Q$. Introducing the variable $u = Ax$, we can rewrite (P) in the form

$$\begin{aligned} \min & - \sum_{i=1}^n \ln x_i \\ \text{s.t.} & \frac{1}{2} \|u\|^2 + \langle b, x \rangle + c \leq 0, \\ & Ax - u = 0. \end{aligned}$$

Using the standard Lagrangian approach, formulate the dual problem explicitly, that is, as a maximization problem in which the objective function is given explicitly.

Proof: We start from:

$$(P) \quad \min_{x>0} - \sum_{i=1}^n \ln(x_i) \quad \text{s.t.} \quad \frac{1}{2} \langle Qx, x \rangle + \langle b, x \rangle + c \leq 0.$$

Since Q is positive definite, we can factor it as $Q = A^T$ for some A symmetric and positive definite. Define $u = Ax$. Then:

$$\frac{1}{2} \langle Qx, x \rangle = \frac{1}{2} \|Ax\|^2 = \frac{1}{2} \|u\|^2.$$

Thus, (P) becomes:

$$\min_{x>0, u} - \sum_{i=1}^n \ln(x_i) \quad \text{s.t.} \quad \frac{1}{2} \|u\|^2 + \langle b, x \rangle + c \leq 0, \quad Ax - u = 0.$$

Introducing Lagrange multipliers $\lambda \geq 0$ for the inequality constraint and $y \in \mathbb{R}^n$ for the equality constraint $Ax - u = 0$ gives the Lagrangian:

$$L(x, u, \lambda, y) = - \sum_{i=1}^n \ln(x_i) + \lambda \left(\frac{1}{2} \|u\|^2 + b^T x + c \right) + y^T (Ax - u).$$

First, minimize w.r.t. u :

$$\frac{\partial L}{\partial u} = \lambda u - y = 0 \implies u = \frac{y}{\lambda}.$$

Substitute back into L :

$$L(x, \lambda, y) = - \sum_{i=1}^n \ln(x_i) + \lambda (b^T x + c) + y^T Ax + \lambda \left(\frac{1}{2} \frac{\|y\|^2}{\lambda^2} \right) - y^T \left(\frac{y}{\lambda} \right).$$

Since

$$\frac{1}{2} \frac{\|y\|^2}{\lambda} - \frac{\|y\|^2}{\lambda} = - \frac{\|y\|^2}{2\lambda}.$$

Then

$$L(x, \lambda, y) = - \sum_{i=1}^n \ln(x_i) + \lambda (b^T x + c) + y^T Ax - \frac{\|y\|^2}{2\lambda}.$$

Next, minimize w.r.t. x .

$$\frac{\partial L}{\partial x_i} = - \frac{1}{x_i} + \lambda b_i + (A^T y)_i = 0 \implies x_i = \frac{1}{\lambda b_i + (A^T y)_i}.$$

Substitute x_i into the Lagrangian:

$$- \sum_{i=1}^n \ln x_i = \sum_{i=1}^n \ln(\lambda b_i + (A^T y)_i).$$

Also:

$$b^T x = \sum_{i=1}^n \frac{b_i}{\lambda b_i + (A^T y)_i}, \quad Ax = A \left(\frac{1}{\lambda b + A^T y} \right).$$

Thus, the dual function $d(\lambda, y)$ is:

$$d(\lambda, y) = \sum_{i=1}^n \ln(\lambda b_i + (A^T y)_i) + \lambda \left(c + \sum_{i=1}^n \frac{b_i}{\lambda b_i + (A^T y)_i} \right) + y^T A \left(\frac{1}{\lambda b + A^T y} \right) - \frac{\|y\|^2}{2\lambda}.$$

The dual problem is:

$$(D) \quad \max_{\lambda \geq 0, y \in \mathbb{R}^n} d(\lambda, y). \quad \square$$

- (c) Suppose an optimal solution to the dual problem is somehow found. Use this knowledge to determine the optimal solution $x^* \in \mathbb{R}^n$ to the original problem (P) .

Proof: Using the Theorem 11.5,

$$x_i^* = \frac{1}{\lambda^* b_i + (A^T y^*)_i}$$

is optimal primal solution x^* in terms of the dual solution.

- (d) Using the above approach, formulate the dual problem when (P) has more than one ($k > 1$) convex quadratic inequality.

Proof: If we have $k > 1$ convex quadratic inequalities:

$$\frac{1}{2}\langle Q_j x, x \rangle + b_j^\top x + c_j \leq 0, \quad j = 1, \dots, k,$$

we introduce $u_j = A_j x$ where $Q_j = A_j^2$. The constraints become:

$$\frac{1}{2}\|u_j\|^2 + b_j^\top x + c_j \leq 0, \quad A_j x - u_j = 0, \quad j = 1, \dots, k.$$

Introducing $\lambda_j \geq 0$ and y_j for each constraint:

$$L(x, \{u_j\}, \{\lambda_j\}, \{y_j\}) = -\sum_{i=1}^n \ln(x_i) + \sum_{j=1}^k \lambda_j \left(\frac{1}{2}\|u_j\|^2 + b_j^\top x + c_j \right) + \sum_{j=1}^k y_j^\top (A_j x - u_j).$$

Following the same steps as the single-constraint case, we get a dual problem in terms of (λ_j, y_j) for $j = 1, \dots, k$. The structure remains similar, but each constraint contributes an additional pair (λ_j, y_j) and corresponding terms in the dual objective. $\square$

Part II

Using the same vectors c and a_j from the first homeworks, we want to solve the following linear problem. Recall that there are $m = 2000$ constraints with $\|a_j\| = 1$, and x has $n = 5000$ dimensions.

$$\begin{aligned} \min \quad & \langle c, x \rangle \\ \text{s.t.} \quad & \langle a_j, x \rangle \leq 1 \quad \forall j \\ & |x_i| \leq 1 \quad \forall i \end{aligned}$$

We will consider three methods to solve this problem. For each method, allocate the same amount of time and report the optimality gap (estimate f^* as accurately as possible, potentially using an existing solver) and the feasibility gap, i.e., how far the obtained solution is from being a feasible solution. For the first two methods, you must solve the unconstrained minimization problems using one of the following methods:

1. Gradient method with a constant step size.
2. Gradient method with backtracking.
3. Gradient method with backtracking and one of the Goldstein or Wolfe conditions.
4. Newton's method.
5. Quasi-Newton BFGS method with backtracking and Wolfe conditions for step size selection.
6. Newton's trust-region method and the dogleg method to approximate the solution of the subproblem.
7. Nesterov's accelerated gradient method with backtracking.
8. Stochastic gradient method with mini-batches.

The methods are as follows:

1. Choose the “best” of the methods mentioned above and use it to solve the linear problem using a barrier-type method. Clearly describe the method: how to update the parameters, stopping criteria, optimality criteria for each subproblem, etc.
2. Choose one of the methods mentioned above to optimize the function $t\langle c, x \rangle - \sum_{j=1}^m \log(1 - \langle a_j, x \rangle)$ and use it to solve the linear problem using a barrier-type method, projecting the solution onto the hypercube $\{x : |x_i| \leq 1, \forall i\}$ at each iteration. Clearly describe the method: how to update the parameters, stopping criteria, optimality criteria for each subproblem, etc.

Method	Approach	Final Value	Gap to Optimal	Feasibility Gap
Method A	Constant Step	-9.41646706	1214.9785	0.0
Method B	Backtracking	-306.5924290	917.802623	0.0
Method C	Nesterov + Backtracking	-1223.711926	0.6831260	1.0425014
Barrier Method	Projected gradient with backtracking	-375.247409	849.14764	0.0

Table 1: Results for the different methods

Solution: By making use of `scipy.optimize.linprog` the estimated optimal value is $f^* = -1224.395052671263$.

For the first problem we tried gradient descent with constant step size, gradient descent with backtracking, and Nesterov's accelerated gradient method with backtracking. For the second problem, we used projected gradient with backtracking. The results are shown in the table below. 10 seconds were allocated for each method.

```

1 import numpy as np
2 import pandas as pd
3 import time
4 from scipy.optimize import linprog
5
6 np.random.seed(3527)
7
8 # Problem dimensions
9 n = 5000
10 m = 2000
11
12 df = pd.read_csv('A.txt', header=None)
13 A = df.values
14
15 # Generate c between -0.5 and 0.5
16 c = np.random.rand(n) - 0.5
17
18 # Estimate f* using scipy
19 bounds = (-1, 1)
20 res = linprog(c, A_ub=A, b_ub=np.ones(m), bounds=[bounds]*n, method='highs')
21 f_star = res.fun
22 print("Estimated f* =", f_star)
23
24 #####
25 # Helper functions
26 #####
27
28 def project_onto_box(x, lower=-1.0, upper=1.0):
29     return np.clip(x, lower, upper)
30
31 def feasibility_gap(x, A):
32     # Measures how much constraints are violated:
33     # max(a_j^T x - 1, 0) and max(|x_i|-1,0)
34     Ax = A.dot(x)
35     cons_viol = np.maximum(Ax - 1, 0).sum()
36     box_viol = np.sum(np.maximum(np.abs(x)-1,0))
37     return cons_viol + box_viol
38
39 def objective_value(x, c):
40     return np.dot(c, x)
41
42 def penalty_function(x, c, A, mu=1000.0):
43     # Unconstrained penalty form:
44     # f(x) = <c,x> + mu/2 * sum_j max(a_j^T x -1, 0)^2 + mu/2 * sum_i max(|x_i|-1,0)^2
45     Ax = A.dot(x)
46     viol_linear = np.maximum(Ax-1,0)
47     viol_box = np.maximum(np.abs(x)-1,0)
48     return np.dot(c, x) + (mu/2.0)*(np.sum(viol_linear**2) + np.sum(viol_box**2))
49
50 def penalty_gradient(x, c, A, mu=1000.0):
51     # Gradient of penalty_function
52     Ax = A.dot(x)
53     viol_linear = Ax - 1
54     mask_linear = (viol_linear > 0)
55     grad_linear = A[mask_linear,:].T.dot(viol_linear[mask_linear]) # sum over violating
56     constraints

```

```

56     diff_x = np.abs(x)-1
57     mask_box_pos = (x > 1)
58     mask_box_neg = (x < -1)
59     # gradient w.r.t. box constraints
60     # d/dx_i (max(|x_i|-1,0)^2) = 2(|x_i|-1)*sign(x_i) if violated
61     grad_box = np.zeros_like(x)
62     grad_box[mask_box_pos] = mu*(x[mask_box_pos]-1)
63     grad_box[mask_box_neg] = mu*(x[mask_box_neg]+1)
64
65     return c + mu*grad_linear + grad_box
66
67
68 def report_solution(x, f_star, A, c):
69     obj = objective_value(x, c)
70     opt_gap = obj - f_star
71     feas_gap = feasibility_gap(x, A)
72     return obj, opt_gap, feas_gap
73
74 #####
75 # 1. Three methods for the linear problem
76 #####
77
78 # We implement each method to run for a fixed amount of time and then report results.
79
80 def gradient_constant_step(x0, c, A, mu=1000.0, step=1e-5, max_time=10.0):
81     x = x0.copy()
82     start = time.time()
83     while time.time() - start < max_time:
84         grad = penalty_gradient(x, c, A, mu)
85         x -= step*grad
86     return x
87
88 def gradient_backtracking(x0, c, A, mu=1000.0, alpha_init=1e-3, beta=0.5, gamma=1e-4, max_time=10.0):
89     x = x0.copy()
90     start = time.time()
91     while time.time() - start < max_time:
92         grad = penalty_gradient(x, c, A, mu)
93         fx = penalty_function(x, c, A, mu)
94         step = alpha_init
95         # Backtracking line search
96         while True:
97             x_new = x - step*grad
98             f_new = penalty_function(x_new, c, A, mu)
99             if f_new <= fx - gamma*step*np.dot(grad, grad):
100                 break
101             step *= beta
102             if step < 1e-20:
103                 break
104         x = x_new
105     return x
106
107 def nesterov_accelerated_backtracking(x0, c, A, mu=1000.0, alpha_init=1e-3, beta=0.5, gamma=1e-4, max_time=10.0):
108     x = x0.copy()
109     y = x0.copy()
110     t_old = 1.0
111     start = time.time()
112     while time.time() - start < max_time:
113         grad = penalty_gradient(y, c, A, mu)
114         fy = penalty_function(y, c, A, mu)
115         step = alpha_init
116         # Backtracking line search
117         while True:
118             x_new = y - step*grad
119             f_new = penalty_function(x_new, c, A, mu)
120             if f_new <= fy - gamma*step*np.dot(grad, grad):
121                 break
122             step *= beta
123             if step < 1e-20:
124                 break
125         x_prev = x
126         x = x_new
127         t_new = (1 + np.sqrt(1+4*t_old*t_old))/2
128         y = x + (t_old-1)/t_new * (x - x_prev)

```

```

129         t_old = t_new
130         return x
131
132 # Run and compare
133 x0 = np.zeros(n)
134 time_budget = 10.0
135
136 x_const = gradient_constant_step(x0, c, A, mu=1000.0, step=1e-5, max_time=time_budget)
137 x_back = gradient_backtracking(x0, c, A, mu=1000.0, alpha_init=1e-3, max_time=time_budget)
138 x_nest = nesterov_accelerated_backtracking(x0, c, A, mu=1000.0, alpha_init=1e-3, max_time=
    time_budget)
139
140 print("Method A (Constant Step) Results:", report_solution(x_const, f_star, A, c))
141 print("Method B (Backtracking) Results:", report_solution(x_back, f_star, A, c))
142 print("Method C (Nesterov + Backtracking) Results:", report_solution(x_nest, f_star, A, c))
143
144 #####
145 # 2. Barrier-type method
146 #####
147
148 # Solve:
149 #  $\min t \langle c, x \rangle - \sum_j \log(1 - a_j^T x)$ 
150 # subject to  $|x_i| \leq 1$ 
151 #
152 # We'll use gradient method with backtracking and projection to solve subproblems.
153 # Increase t iteratively.
154 # Stop when difference from f_star is small or feasibility gap is small.
155
156 def barrier_objective(x, c, A, t):
157     Ax = A.dot(x)
158     # If outside domain, return infinity
159     if np.any(Ax >= 1):
160         return np.inf
161     return t*np.dot(c, x) - np.sum(np.log(1 - Ax))
162
163 def barrier_gradient(x, c, A, t):
164     Ax = A.dot(x)
165     denom = 1 - Ax
166     # If outside domain, gradient is not defined properly
167     if np.any(denom <= 0):
168         return np.full_like(x, np.inf)
169     grad = t*c + np.sum(A.T/(denom), axis=1)
170     return grad
171
172 def solve_barrier_subproblem(x0, c, A, t, alpha_init=1e-3, beta=0.5, gamma=1e-4, max_iter=500,
    tol=1e-6):
173     x = x0.copy()
174     for _ in range(max_iter):
175         grad = barrier_gradient(x, c, A, t)
176
177         # If gradient is infinite, we are outside domain break
178         if np.any(np.isinf(grad)):
179             break
180
181         gnorm = np.linalg.norm(grad)
182         if gnorm < tol:
183             break
184
185         fx = barrier_objective(x, c, A, t)
186         step = alpha_init
187
188         # Add a maximum number of backtracking attempts
189         max_backtracking = 50
190         backtrack_count = 0
191         success = False
192
193         while backtrack_count < max_backtracking:
194             x_new = project_onto_box(x - step*grad)
195             f_new = barrier_objective(x_new, c, A, t)
196
197             if f_new == np.inf:
198                 step *= beta
199                 backtrack_count += 1
200                 continue
201

```



```

202         # Armijo condition
203         if f_new <= fx - gamma * step * np.dot(grad, grad):
204             success = True
205             x = x_new
206             break
207
208         step *= beta
209         backtrack_count += 1
210
211     if not success:
212         break
213
214     return x
215
216 def barrier_method(c, A, x0=None, t0=1.0, mu=10.0, outer_max_iter=10, inner_tol=1e-6):
217     if x0 is None:
218         x0 = np.zeros_like(c)
219     x = x0.copy()
220     for k in range(outer_max_iter):
221         x = solve_barrier_subproblem(x, c, A, t0, max_iter=1000, tol=inner_tol)
222         obj = np.dot(c, x)
223         feas = np.sum(np.maximum(A.dot(x)-1,0)) + np.sum(np.maximum(np.abs(x)-1,0))
224         if feas < 1e-6:
225             break
226         t0 *= mu
227     return x
228
229 # Run and compare
230 x_barrier = barrier_method(c, A, t0=1.0, mu=10.0, outer_max_iter=10, inner_tol=1e-6)
231 print("Barrier Method Results:", report_solution(x_barrier, f_star, A, c))

```